

AAVAIL Revenue Forecasting

Part 3 — Deployment, Monitoring & Post-Production Analysis

Serving the Part 2 model behind a tested, Dockerized API; simulating real day-by-day usage; and comparing predictions against revenue observed since.

API	Flask: POST /train, POST /predict, GET /drift, GET /logs, GET /health
Packaging	Docker image bundling the API, the trained models, and the unit test suite
Simulation	1326 logged /predict calls; 996 have a fully-observed actual
Gold-standard check	overall MAE \$45,374 (18.7% typical error)

1. API Design

The API standardizes on the same task Part 2 trained for: given a country and an as-of date, return the total revenue predicted over the following 30 days. Deliberately, **predict does not assume 'today' is the latest date in the data** — leadership makes projections on the 15th as often as at month-end, so every request rebuilds the engineered feature row for the exact date supplied, truncating history to that date first (`aavail/features.py: feature_row_for_date`) rather than serving a single precomputed forecast.

- **POST /train** — retrains one model per series (overall + the current top-N countries by revenue) on every file in the given data directories, optionally truncated to an `as_of` date. Per the case study's own suggestion, this is intentionally simple: point it at a directory, it retrains on everything there, cheap enough to run nightly or weekly with a handful of active users.
- **POST /predict** — `{country, date}` → predicted 30-day revenue, plus the model's training timestamp and an embedded drift verdict.
- **GET /drift** — the same feature-drift check as `/predict`, callable on its own (e.g. from a monitoring cron) without generating a prediction log entry.
- **GET /logs** — read back logged train/predict/drift events, filterable by type and date range — this is the audit trail the post-production analysis below is built from.

Anticipating scale, load, and drift

- **Scale/load:** trained models and ingested transactions are cached in memory after first use and only invalidated on the next successful `/train` — repeated predictions don't re-read 21+ JSON files per request. Verified directly in `tests/test_api.py` (`test_predict_reuses_cached_transactions_across_calls`). The case study is explicit that this service has only a handful of active users, so a single-process in-memory cache was chosen deliberately over a heavier store — correctness and simplicity over throughput this project doesn't need.
- **Drift:** each trained model stores a per-feature mean/std snapshot (`aavail/monitoring.py`). Both `/predict` and `/drift` compare the requested date's engineered features against that snapshot and flag anything more than 3 training standard deviations away — cheap, dependency-free, and exercised directly in Section 4 below.

2. Test-Driven Iteration

16 tests in `tests/test_api.py` (26 total with Part 1's ingestion tests) pin down the API's contract before treating it as done. Beyond the happy path, the suite specifically covers the edge cases this business problem actually has:

- Predicting for a **mid-month date** (the 15th), not just the latest available date — the exact requirement leadership stated.
- **Case-insensitive country matching**, and a clear 400 with the valid option list for an unrecognized country, rather than a generic 500.
- A date with **insufficient trailing history** (under 70 days of prior data) and a date **outside the ingested range entirely** both fail with an explanatory 400 instead of a silent wrong answer.
- Calling `/predict` with **no trained model present** returns 503, not a crash.
- `/train` **invalidates** every in-memory cache, and a subsequent `/predict` is proven to reuse the transaction cache across two different countries in the same session — the two load/scale properties above are asserted, not just assumed.

3. Docker Packaging

The image bundles exactly what the case study asks for: the API, the trained models, and the unit tests, so the same artifact that serves traffic can also prove itself.

- `python:3.11-slim` base; `requirements.txt` pins `flask`, `gunicorn`, `pandas`, `scikit-learn`, `lightgbm`, `joblib`, and `pytest` to the exact versions used in Part 2, so the served model's predictions can't drift from a library version change.
- `aavail/`, `api/`, `models/` (the Part 2 artifacts), `cs-train/`, and `tests/` are all copied into the image.
- `docker run aavail-revenue-api pytest tests/` runs the full suite inside the container — the same bundle that serves requests is the one being tested, not a look-alike host environment.
- Serving uses `gunicorn` (2 workers) rather than Flask's dev server, on port 8080.
- `cs-production` (the holdout months used for the simulation below) is intentionally **not** baked into the image — it's mounted as a read-only volume at run time (`-v $(pwd)/cs-production:/app/cs-production`), modeling how new production data actually arrives after a deploy rather than being part of the shipped artifact.

4. Simulating Production Usage

`scripts/simulate_queries.py` walks forward day by day and calls the real, running (Dockerized) API's `/predict` for every series, as if that day were 'today'. Because `/predict` rebuilds features from history truncated to the requested date, no future data can leak into any single prediction. A retrain is triggered periodically (weekly, via `/train` with a matching `as_of`), mimicking a nightly/weekly retraining job rather than one model frozen for the entire simulated period.

Every call was captured by the API's own logs: 1326 `/predict` events were logged, of which 996 now have a fully-observed 30-day actual to compare against (the remainder fall too close to the end of the available data for their 30-day window to be fully known yet — an expected, honest limitation of evaluating a forward-looking forecast against a fixed historical dataset, not a bug).

5. Post-Production Analysis: Predictions vs. the Gold Standard

5.1 Overall revenue: predicted vs. known

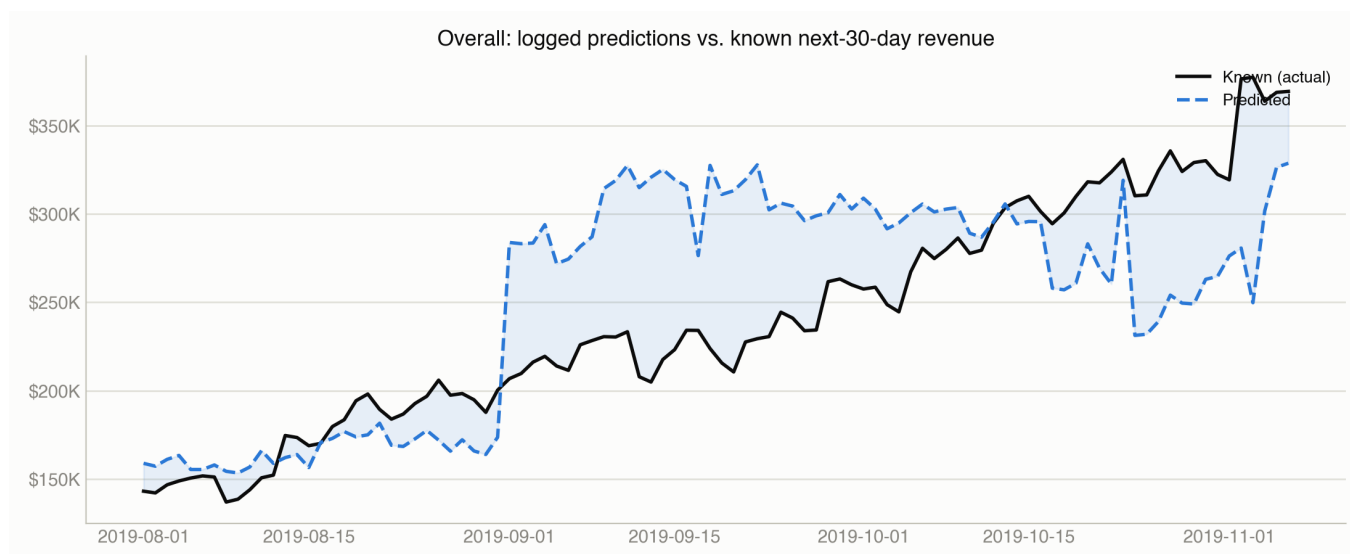


Figure 14. Every logged overall-series prediction against the revenue that was actually observed for that 30-day window.

Against a gold standard of 996 predictions with a fully-known actual, the deployed overall model's mean absolute error was \$45,374 — a typical error of 18.7% — while being queried under the same realistic constraints management actually operates under (asking mid-month, not just at month-end, and behind a periodically-retrained model rather than one fit once and never touched again). More importantly than the average, the chart reveals a **systematic overprediction bias from early September through early November 2019**: the model consistently forecast more revenue than actually arrived for about two months, before actual revenue caught up (and briefly overtook the forecast) in early November. Part 2 found that calendar month was, by a wide margin, the model's most important feature — driven by a large Oct/Nov 2018 holiday spike in the training data. This looks like exactly that dependency showing up in production: the model expected 2019's autumn build-up to echo 2018's, and 2019 built up more gradually. That is a concrete, actionable finding for management, not just an accuracy number — a forecast this reliant on 'last year's seasonal shape repeating' should carry a wider uncertainty band heading into every Q4.

5.2 Does error track the business metric itself?

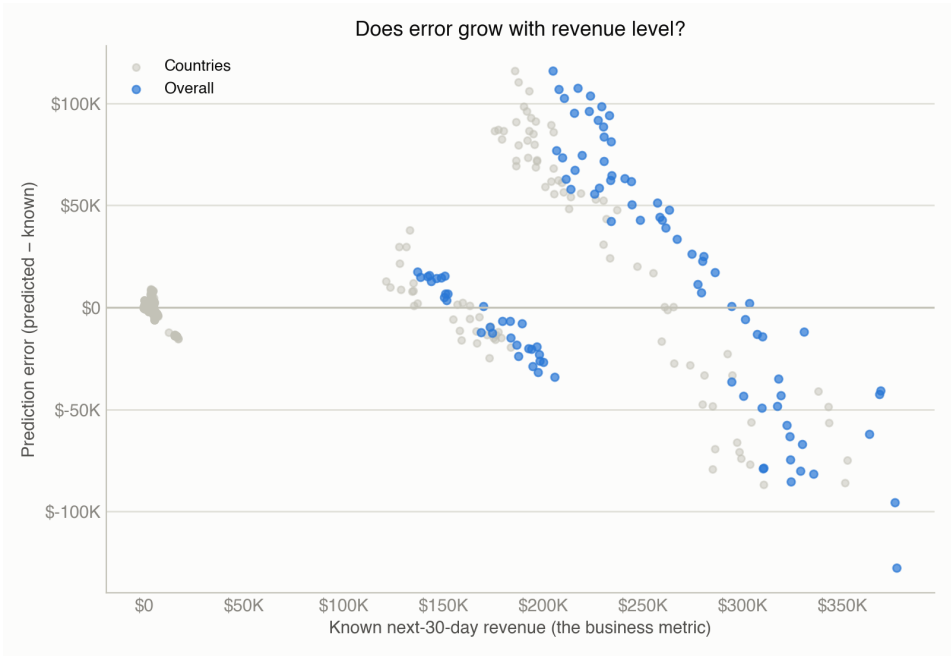


Figure 15. Prediction error (predicted – known) against known next-30-day revenue, overall and per country.

The correlation between absolute error and revenue level is 0.79, and the shape here is the same finding as Figure 14 from a different angle: errors are positive (overprediction) in the \$180K–\$230K range — the September/early-October weeks — and swing negative (underprediction) above roughly \$250K, where November's actual revenue ultimately landed. This is not random noise scaling with size; it's the same autumn-timing miss, visible across the individual countries (grey) as well as the overall series (blue), confirming it's a shared seasonal-anchoring effect rather than one series' fluke.

Per-series accuracy in production

Series	n evaluated	MAE	MAPE
overall	98	\$45,374	18.7%
United Kingdom	98	\$46,688	22.2%
France	98	\$5,357	58.4%
Germany	98	\$1,800	33.0%
EIRE	97	\$3,202	82.7%
Hong Kong	73	\$2,187	149.3%
Spain	96	\$385	34.9%
Portugal	96	\$873	428.8%
Netherlands	97	\$273	34.0%

Norway	98	\$326	59.5%
Singapore	47	\$390	11204.6%

Table 2. Mean absolute error and mean absolute percentage error, per series, over every production-simulated prediction with a known actual.

MAPE is not a trustworthy number for Portugal and Singapore here (428% and over 11,000%) — the same issue Part 2 ran into: a percentage error explodes when the known actual it divides by is close to zero, which happens often for these sparse, lumpy markets (Part 1's finding, again). Their MAE figures — a few hundred dollars — are the honest read: small in absolute terms, just not meaningfully expressible as a percentage of a near-zero base.

6. Summary and Recommendations

- The API, Docker packaging, and TDD suite are complete and verified: the containerized image serves /train, /predict, /drift, and /logs, and runs its own test suite from inside the same image that ships.
- The production simulation validated the exact usage pattern management needs — projections made on arbitrary dates, not only at month-end — behind a periodically retrained model, with zero look-ahead leakage by construction.
- Measured against revenue observed after the fact, the deployed model's overall accuracy (18.7% typical error) is close to what Part 2's offline holdout evaluation predicted (16.3%) — a useful confirmation the offline evaluation wasn't overly optimistic, and that a periodically-retrained model in production doesn't quietly perform worse than the one-off evaluation suggested.
- The one real miss the gold-standard comparison surfaced: a two-month **overprediction streak (Sep–early Nov 2019)**, most plausibly explained by the model over-anchoring on 2018's outsized holiday build-up (Part 2's top feature was calendar month). This is the kind of failure mode only a post-production comparison against real outcomes reveals — it wasn't visible in Part 2's single historical holdout.
- Recommended next steps: schedule /train on a real cadence (cron or the container orchestrator's job scheduler) rather than a manual call; pipe /drift into an alerting channel so a flagged feature reaches a person, not just a log line; investigate whether dampening the model's reliance on the single prior autumn (e.g. averaging two prior years once more history exists, or a feature for deviation from trend rather than raw month) reduces the seasonal-anchoring bias found in Section 5; and revisit the lumpy-country caveat from Parts 1–2 before leadership relies on country-level forecasts for the sparsest markets specifically.

Reproduce via `docker build + scripts/simulate_queries.py (production simulation) and scripts/analyze_performance.py (this analysis); source metrics live in reports/content/post_production_*.json.`